# C# Roadmap: 12-Week Plan

October 9 to December 31, 2026: from complete beginner to a deployed, portfolio-ready project

You have 12 weeks (84 days). This plan compresses the full C# journey into that window by focusing on the essentials in each phase and ending with a capstone project: a tested, deployed web API. Deeper expert-level topics are pushed to a clearly marked 'after December 31' phase.

> **A realistic expectation:** in 12 weeks you can become a confident, job-ready **junior to early-intermediate** C# developer. True expert level (architecture, performance tuning, runtime internals) typically takes years of real-world practice. This plan gets you the strongest possible foundation and starts you on that path.

**Time commitment assumed:** about 10 to 12 hours per week (roughly 1.5 hours a day, or longer sessions on weekends). If you can do more, spend the extra time building projects, not watching more tutorials.

## The journey at a glance

| Phase | Focus | Dates | Duration |
|---|---|---|---|
| **0** | Setup and mindset | Oct 9 - Oct 10 | 2 days |
| **1** | C# fundamentals | Oct 11 - Oct 29 | ~2.5 weeks |
| **2** | Object-oriented programming | Oct 30 - Nov 12 | 2 weeks |
| **3** | Intermediate C# (collections, LINQ, testing) | Nov 13 - Dec 3 | 3 weeks |
| **4** | Async, DI and design essentials | Dec 4 - Dec 17 | 2 weeks |
| **5** | Capstone: ASP.NET Core Web API | Dec 18 - Dec 31 | 2 weeks |
| **6** | Expert path (continues in 2027) | Jan 2027 onward | Ongoing |

## Golden rules for a short timeline

- **Code every single day.** With only 12 weeks, consistency is everything.
- **Aim for 70% building, 30% learning.** Tutorials only help if you apply them right away.
- **Type everything yourself.** Never copy-paste from tutorials.
- **Do not get stuck for more than 30 to 45 minutes.** Search, ask, or move on and return later.
- **Use Git from day one.** Commit every project to GitHub as your portfolio.
- **Protect the capstone.** If you fall behind, trim stretch topics, never the final project.

# Week-by-week schedule

| Week | Dates | Focus | Deliverable |
|---|---|---|---|
| **1** | Oct 9 - 15 | Setup, variables, types, input/output, conditions | Hello World + number-guessing game on GitHub |
| **2** | Oct 16 - 22 | Loops, methods, arrays, List<T>, strings | Console calculator with menu |
| **3** | Oct 23 - 29 | Exceptions, debugging, nullability, mini-project | Console to-do list app |
| **4** | Oct 30 - Nov 5 | Classes, objects, properties, encapsulation | Bank account system (v1) |
| **5** | Nov 6 - 12 | Inheritance, polymorphism, interfaces, records, enums | Library or inventory app (OOP design) |
| **6** | Nov 13 - 19 | Collections, generics, file I/O, JSON | Contact book that saves to a file |
| **7** | Nov 20 - 26 | LINQ, lambdas, delegates, events | Data analyzer (CSV/JSON + LINQ) |
| **8** | Nov 27 - Dec 3 | Unit testing (xUnit), SQL basics, Git workflow | Add a test suite to a previous project |
| **9** | Dec 4 - 10 | async/await, Tasks, intro to dependency injection | Async file downloader with cancellation |
| **10** | Dec 11 - 17 | SOLID, core design patterns, refactoring | Refactor a past project using DI + SOLID |
| **11** | Dec 18 - 24 | ASP.NET Core Web API, EF Core, database | Working API with CRUD endpoints |
| **12** | Dec 25 - 31 | Auth, validation, tests, Docker, deploy, README | Capstone deployed + documented |

Tip: the schedule has no spare weeks, so use weekends to catch up if a week slips. See 'If you fall behind' near the end for what to trim.

## Phase 0: Setup and Mindset

Oct 9 - Oct 10 (Days 1-2) | Goal: have a working environment and run your first program

### What to learn

- Install the latest .NET SDK from Microsoft
- Choose an editor: Visual Studio (Windows), VS Code with the C# Dev Kit, or Rider
- Learn the `dotnet` CLI: `new`, `build`, `run`
- Git basics: init, add, commit, push; create a GitHub account

### Build these

- Hello World console app
- Program that asks your name and greets you

> **Milestone:** You can create, run and commit a console project without help.

## Phase 1: C# Fundamentals

Oct 11 - Oct 29 (~2.5 weeks) | Goal: think in code and write small programs confidently

### What to learn

- Variables, data types, type conversion, string interpolation
- Operators and expressions
- Control flow: if/else, switch, for, foreach, while
- Methods: parameters, return values, overloading
- Arrays and `List<T>` basics
- Exceptions: try/catch/finally
- Debugging with breakpoints; null handling basics

### Build these

- Number-guessing game
- Calculator with a menu
- Console to-do list
- Unit converter (stretch)

> **Milestone:** You can solve beginner challenges (FizzBuzz, palindromes, simple sorting) without looking up the answer.

## Phase 2: Object-Oriented Programming

Oct 30 - Nov 12 (2 weeks) | Goal: model real-world problems with classes and objects

### What to learn

- Classes, objects, fields, properties, constructors
- Access modifiers, static members, namespaces
- Encapsulation, inheritance, polymorphism, abstraction
- Interfaces vs abstract classes
- Records, enums, value vs reference types
- Clean-code habits: naming, small methods, single responsibility

### Build these

- Bank account system (accounts, transactions, errors)
- Library or inventory management app

> **Milestone:** You can sketch a small class design before coding and explain why you chose an interface or inheritance.

## Phase 3: Intermediate C#

Nov 13 - Dec 3 (3 weeks) | Goal: write idiomatic C# and work with real data

### What to learn

- Collections: List, Dictionary, HashSet, Queue, Stack
- Generics and extension methods
- **LINQ**: Where, Select, GroupBy, OrderBy, Join
- Delegates, events, lambdas, `Func` and `Action`
- File I/O and JSON (System.Text.Json); pattern matching
- Unit testing with xUnit (arrange-act-assert)
- SQL basics: SELECT, INSERT, UPDATE, DELETE, JOIN

### Build these

- Contact book saving to a file
- CSV/JSON data analyzer using LINQ
- Test suite for one of your earlier projects

> **Milestone:** You reach for LINQ naturally and your code has automated tests.

## Phase 4: Async, DI and Design Essentials

Dec 4 - Dec 17 (2 weeks) | Goal: learn the concepts every professional C# codebase relies on

### What to learn

- **async/await**, Task, cancellation tokens, common pitfalls
- Garbage collection and IDisposable / using (conceptual understanding)
- Dependency injection and inversion of control
- SOLID principles
- Core patterns: Strategy, Factory, Repository, Observer
- Stretch: threading basics, reflection, Span<T>

### Build these

- Async file downloader with progress and cancellation
- Refactor an earlier project to use DI and SOLID

> **Milestone:** You can explain how async/await works and justify your design choices.

## Phase 5: Capstone: ASP.NET Core Web API

Dec 18 - Dec 31 (2 weeks) | Goal: ship one deployed, tested, documented project

### What to learn

- ASP.NET Core Web API: controllers or minimal APIs, routing, model binding
- Entity Framework Core: models, migrations, relationships
- Validation, error handling, basic authentication (JWT)
- Integration with a database (SQLite for development, PostgreSQL or SQL Server for deploy)
- Docker basics and deploying to a free or low-cost cloud host
- Writing a clear README with setup steps and decisions

### Build these

- **Capstone:** a REST API such as a task manager, expense tracker or mini e-commerce backend, with authentication, a database, tests and Docker
- Prefer desktop, games or mobile? Swap this phase for WPF, Unity or .NET MAUI with the same 2-week capstone idea

> **Milestone:** You have one public, working project on GitHub that you can show in interviews.

## Phase 6: Expert Path (January 2027 onward)

Ongoing | Goal: turn a strong foundation into real expertise

### What to learn

- **Architecture:** Clean Architecture, DDD, CQRS, microservices vs modular monoliths
- **Performance:** profiling, BenchmarkDotNet, allocation analysis, caching
- **Quality:** integration tests, observability, security best practices
- **Runtime internals:** JIT, GC modes, how async and LINQ compile
- **Cloud and DevOps:** CI/CD, Azure or AWS, infrastructure as code
- **Craft:** system design, code review, mentoring, open-source contributions

### Build these

- Contribute to an open-source .NET project
- Build a second, more complex system (messaging, caching, resilience)
- Apply for junior or intern roles while continuing to learn

> **Milestone:** You make sound design decisions and can diagnose problems in a real production system. This usually takes 1 to 3 years of practice.

# Recommended resources

| Type | Resource | Best for |
|---|---|---|
| **Official docs** | Microsoft Learn and the C# documentation | Free, accurate, up to date; use throughout |
| **Practice** | Exercism (C# track), LeetCode, Advent of Code | Problem solving (Phases 1-3) |
| **Book** | C# in Depth by Jon Skeet | Deep language understanding (Phase 3 onward) |
| **Book** | CLR via C# by Jeffrey Richter | How the runtime works (Phase 6) |
| **Book** | Clean Code and Clean Architecture by Robert C. Martin | Design principles (Phases 4-6) |
| **Community** | Official .NET blog, GitHub dotnet repos, r/csharp | Staying current and getting help |

# If you fall behind

- **Cut stretch items first** (marked 'stretch' in the phases).
- **Never skip fundamentals or OOP.** Everything later depends on them.
- **Shorten Phase 4** to async/await and DI only; patterns can wait until January.
- **Simplify the capstone** (fewer endpoints, skip Docker) rather than abandoning it.

# Your first 7 days (Oct 9 - Oct 15)

- Oct 9: install .NET and your editor; run Hello World.
- Oct 10: set up Git and GitHub; push your first project.
- Oct 11: variables, types and user input.
- Oct 12: if/else and comparison logic with 3 tiny exercises.
- Oct 13: loops; practice with FizzBuzz and a multiplication table.
- Oct 14: build the number-guessing game.
- Oct 15: review, note what confused you, and plan Week 2.

Check Microsoft Learn for the latest C# and .NET versions and any newer learning paths, as the ecosystem evolves quickly.